GraphQL basics

11 min read Updated 24 Jul 2026 Topic 68 of 70

Why it matters

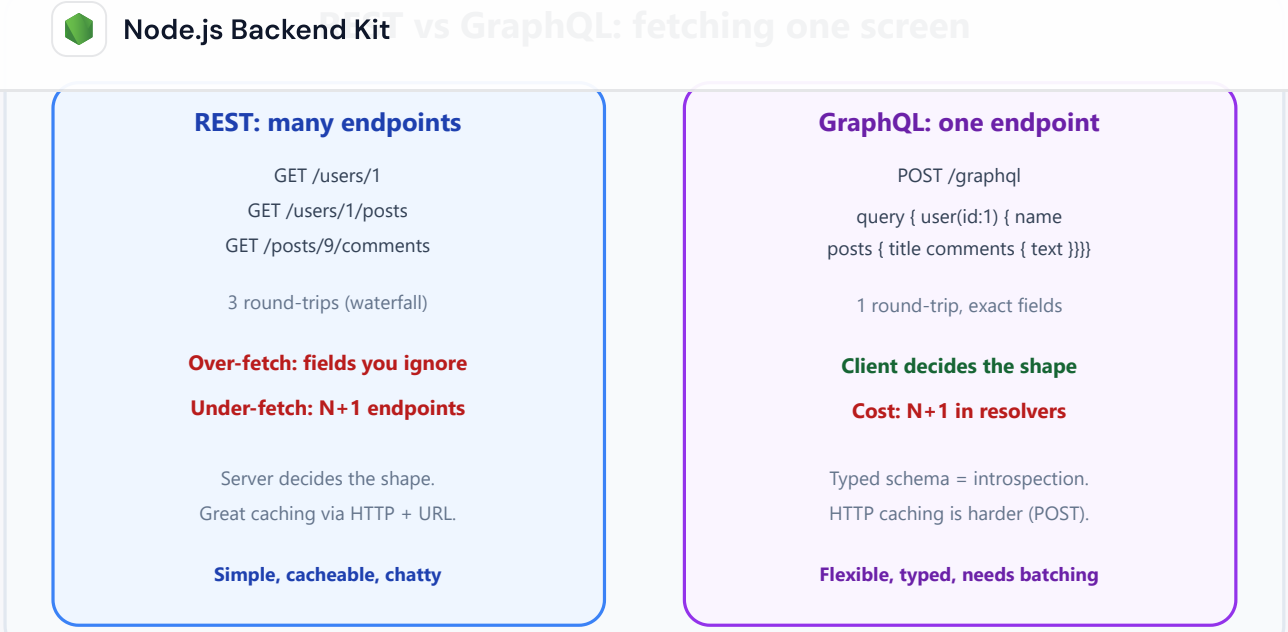
REST organises an API around **resources at URLs**: the server decides what each endpoint returns, and the client takes it or leaves it. That works beautifully until a screen needs a little of this resource and a little of that one. Then you either build a bespoke endpoint for every screen, or you make the client fire several requests and stitch the results together. Mobile teams felt this pain first — every extra round-trip on a flaky network hurts — and GraphQL grew out of it at Facebook.

GraphQL flips the control: the **client sends a query describing exactly the shape it wants**, and the server returns precisely that — no more, no less — from a single endpoint. Understanding GraphQL is now table stakes for backend interviews, not because it replaces REST (it does not), but because knowing *when* its flexibility pays for its added complexity is exactly the kind of trade-off judgement senior engineers are hired for.

The concept, simply

A GraphQL API is defined by a **schema**: a strongly-typed contract listing every type, every field, and the entry points (`Query` for reads, `Mutation` for writes, `Subscription` for real-time streams). The client sends a query that mirrors the shape of the data it wants; the response JSON comes back in that same shape.

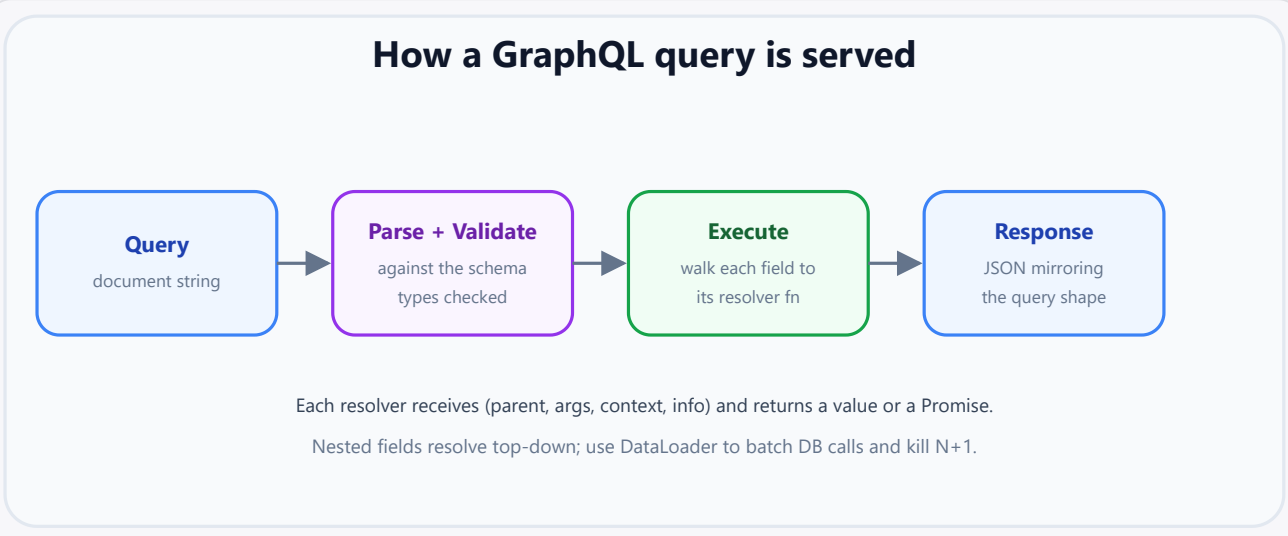
The two problems it targets have names. **Over-fetching** is when an endpoint hands you 40 fields and you needed 3. **Under-fetching** is when one endpoint is not enough, so you call a second and a third — the "waterfall". GraphQL lets one request ask for a user, their last five posts, and each post's comment count, all at once, with every field named explicitly.



REST spreads one screen across several endpoints and fixed payloads; GraphQL lets the client request the exact nested shape from a single endpoint — at the cost of harder caching and resolver-level work.

How it works (deep)

The heart of a GraphQL server is the pairing of a **schema** with **resolvers**. The schema is the "what" — a menu of types and fields. Resolvers are the "how" — a function per field that knows where to get that field's value. When a query arrives, the server does four things: **parses** the query string into an AST, **validates** it against the schema (unknown field? wrong argument type? rejected before any code runs), **executes** by walking the query tree and calling each field's resolver, then assembles the result into JSON matching the requested shape.



Resolution is **hierarchical and top-down**. The `user` resolver runs first and returns a user object (the "parent"). Then, for each field the query asked for on that user, the corresponding resolver runs, receiving that parent. A field resolver's signature is `(parent, args, context, info)`: `parent` is the resolved object above it, `args` are the field's arguments, `context` is a per-request bag (auth, loaders, db handles) shared across every resolver in that request, and `info` is the query AST for advanced cases. If a field maps directly to a property on `parent`, you can omit the resolver — the "default resolver" reads it for you.

This top-down walk creates the single most important GraphQL performance trap: the **N+1 problem**. Ask for 10 posts and each post's author, and the naive author-resolver fires 10 separate database queries — one per post — on top of the 1 query for the posts. The fix is **DataLoader**: a per-request utility that collects all the keys requested within a tick, issues *one* batched query (`WHERE id IN (...)`), and also caches within the request so the same author is never fetched twice. DataLoader is not optional in production GraphQL; it is the thing that makes the flexible query model survivable.

Two more structural facts. First, GraphQL is **transport-agnostic but conventionally a single** `POST /graphql`. That is why HTTP caching (which keys on URL + method) does not help you — every query is a POST to the same URL. Caching moves into the application: persisted queries, response caches keyed on the query, or field-level caching. Second, the schema is **introspectable**: the server can describe its own types, which powers auto-generated docs, IDE autocomplete, and code generation — a genuine developer-experience win REST only gets via extra tooling like OpenAPI.

Code examples

A minimal server with Apollo Server v4 (the current major line) and standalone startup. Schema-first: types in SDL, resolvers as plain functions.

JAVASCRIPT

Copy

```
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';
```

```
  type Post { id: ID!, title: String!, author: User! }
  type Query { user(id: ID!): User, users: [User!]! }
`;

const db = {
  users: [{ id: '1', name: 'Ada' }, { id: '2', name: 'Linus' }],
  posts: [{ id: '9', title: 'Hello', authorId: '1' }],
};

const resolvers = {
  Query: {
    user: (_parent, { id }) => db.users.find(u => u.id === id),
    users: () => db.users,
  },
  User: {
    // resolves the nested "posts" field for a given user parent
    posts: (user) => db.posts.filter(p => p.authorId === user.id),
  },
  Post: {
    author: (post) => db.users.find(u => u.id === post.authorId),
  },
};

const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, { listen: { port: 4000
console.log('ready at ' + url);
```

A query the client would send, and the response shape it produces — identical trees.

JAVASCRIPT Copy

```
// Request body sent to POST /graphql
const query = `
  query GetUser($id: ID!) {
    user(id: $id) {
      name
      posts { title }
    }
  }
```

```
// Response:
// { "data": { "user": { "name": "Ada", "posts": [{ "title": "Hello" }] } }
```

Killing N+1 with DataLoader. One batched lookup per request, plus a per-request cache. The loader lives on `context` so every resolver in the request shares it.

JAVASCRIPT Copy

```javascript
import DataLoader from 'dataloader';

function createContext() {
  const userLoader = new DataLoader(async (ids) => {
    // ONE query for all requested ids
    const rows = await db.usersWhereIdIn(ids); // returns array
    const byId = new Map(rows.map(u => [u.id, u]));
    return ids.map(id => byId.get(id)); // must match input order
  });
  return { userLoader };
}

const resolvers = {
  Post: {
    // 100 posts → 1 batched user query, not 100
    author: (post, _args, ctx) => ctx.userLoader.load(post.authorId),
  },
};

const server = new ApolloServer({ typeDefs, resolvers });
await startStandaloneServer(server, { context: async () => createContext()
```

A mutation with input validation and an auth check pulled from context — writes look just like reads but live under `Mutation`.

JAVASCRIPT Copy

```javascript
const typeDefs = `
  type Post { id: ID!, title: String! }
```

```
const resolvers = {
  Mutation: {
    createPost: async (_p, { input }, ctx) => {
      if (!ctx.userId) throw new Error('UNAUTHENTICATED');
      if (input.title.trim().length === 0) throw new Error('title required
      return db.insertPost({ title: input.title, authorId: ctx.userId });
    },
  },
};
```

Common mistakes & gotchas

- **Ignoring N+1 until it takes production down.** The demo with 3 rows is instant; the real table with nested fields fires hundreds of queries per request. DataLoader (or equivalent batching) is mandatory, not a nicety.
- **Expecting HTTP caching to "just work".** Because everything is one `POST /graphql`, CDNs and browser caches cannot key on it. You must add persisted queries or application-level caching deliberately.
- **Leaving introspection and unbounded queries open in production.** A public schema is a map of your whole domain, and deeply nested queries can be a denial-of-service vector. Add query depth/complexity limits and consider disabling introspection or requiring persisted queries.
- **Returning raw database errors to the client.** GraphQL surfaces errors in an `errors` array; unmapped exceptions leak stack traces and internal shape. Map to safe, coded errors.
- **Putting side effects in query resolvers.** Queries must be read-only and are executed in parallel; anything that mutates belongs under `Mutation`, whose top-level fields run in series.
- **Treating status codes like REST.** A GraphQL response is typically HTTP 200 even when a field errored — the failure is *inside* the body. Clients must inspect the `errors` array, not just the status line.

- **One giant resolver file with DB logic inline.** It becomes untestable. Keep resolvers thin — they should call a service/data layer, not embed SQL.

Best practices

- **Batch every to-many and to-one lookup with DataLoader,** and create the loaders per-request in `context` so caching never leaks across users.
- **Bound query cost:** enforce maximum depth and a complexity budget, and prefer **persisted queries** for public APIs so only vetted queries run.
- **Keep resolvers thin;** push business logic into a service layer so it is testable and reusable outside GraphQL.
- **Design the schema for clients, not tables.** Model the domain (nullable fields chosen deliberately, meaningful enums, input types) rather than mirroring your database rows.
- **Put auth in context and check it in resolvers/directives,** not in a single gateway middleware — field-level access is the whole point.
- **Version by evolution, not URLs:** add fields freely, deprecate with `@deprecated`, and avoid breaking changes — GraphQL is meant to grow without `/v2`.
- **Use it where its shape wins** — many clients with divergent data needs, or a graph of related entities — and stick with REST for simple, cache-heavy, file-ish endpoints.

⭐ Interview lens

> **Q: What problems does GraphQL solve that REST struggles with?** Over-fetching (endpoints returning more than a screen needs) and under-fetching (needing several round-trips to assemble one view). The client specifies the exact fields, nested, in a single request against one endpoint, and the strongly-typed schema gives introspection and codegen for free.

> **Q: Explain the N+1 problem in GraphQL and how you fix it.** Resolvers execute top-down per field, so a list of N parents whose child field each hits the DB produces 1 + N queries. Fix with DataLoader: it collects keys within a tick, issues one batched

Q: Why is HTTP caching harder with GraphQL than REST? REST caches on URL + method; GET `/users/1` is trivially cacheable. GraphQL is conventionally a single `POST /graphql`, so URL-based caches cannot distinguish queries. You move caching into the app: persisted queries, response caching keyed on the query, or field-level caches.

Q: A resolver throws. What HTTP status does the client see? Usually still **200** — GraphQL reports errors in an `errors` array inside the JSON body while returning partial `data`. Clients must check that array, not rely on the status code. This surprises people coming from REST.

Q: What are the arguments to a resolver? `(parent, args, context, info)`. `parent` is the resolved object one level up, `args` the field arguments, `context` the shared per-request object (auth, loaders, db), `info` the query AST. Loaders and the current user live on `context`.

Q: When would you NOT choose GraphQL? Simple CRUD with few clients, endpoints that benefit from HTTP/CDN caching, file uploads/downloads, or teams without the appetite for schema governance and N+1 discipline. GraphQL's flexibility is overhead you should only pay when many clients need divergent, graph-shaped data.

Cheat-card

- **Schema** = typed contract (`Query` / `Mutation` / `Subscription`); **resolvers** = one function per field, signature `(parent, args, context, info)`.
- **Solves:** over-fetching + under-fetching; client picks exact fields from one endpoint.

- **N+1 ➜ DataLoader:** batch keys per tick into one `IN ( ... )` query + per-request cache.
- **Caching:** single `POST /graphql` ➜ no free HTTP caching; use persisted queries / app cache.
- **Errors:** usually HTTP 200 with an `errors` array; queries parallel, mutations serial.
- **Guardrails:** depth/complexity limits, persisted queries, thin resolvers over a service layer; evolve schema instead of versioning URLs.

QUICK CHECK — no marks, just for you.

1. Which two problems is GraphQL specifically designed to solve that REST struggles with?

- ◯ Authentication and rate limiting
- ◯ Database indexing and connection pooling
- ◯ TLS termination and CORS
- ◯ Over-fetching (endpoints returning more than a screen needs) and under-fetching (needing several round-trips to assemble one view)

Check answer

2. What is the N+1 problem in GraphQL and how is it fixed?

- ◯ Too many mutations run in parallel; fixed by serialising them
- ◯ Resolvers execute top-down per field, so N parents each hitting the DB produce 1+N queries; fixed with DataLoader, which batches keys into one IN(...) query and caches per request
- ◯ The schema has N+1 types; fixed by merging types

Check answer

3. A GraphQL resolver throws an error. What HTTP status does the client typically see?

- ○ A 500 Internal Server Error every time a resolver throws
- ○ A 400 Bad Request, since GraphQL validates the query
- ○ Usually still 200 — GraphQL reports the failure in an errors array inside the JSON body (often with partial data); clients must inspect that array, not the status code
- ○ A 404, because the failed field is treated as not found

Check answer

‹ Previous
Real-time with WebSockets

Next ›
Microservices in Node

LEARN MORE

ARTICLES & DOCS

Learn GraphQL - Introduction, Queries & Schema
graphql.org

Getting Started With GraphQL.js (Node.js)
graphql.org

VIDEOS



Learn GraphQL In 22 Minutes (crash course)
SWE with Vivek Bharatha